天枢IoT Flutter低代码工具 - 开发计划

一、项目概述

1.1 项目名称

天枢IoT Studio - Flutter代码生成式低代码开发工具

1.2 项目定位

这是一个**代码生成工具**，而非运行时低代码平台：

- 用户通过拖拽控件设计界面
- 工具生成**可读性高、结构清晰**的Flutter源代码
- 用户可导出代码进行**二次开发和定制**
- 着重支持**移动端**（iOS/Android），同时兼容桌面端

1.3 核心价值

- **快速原型**：拖拽生成基础App框架
- **代码可控**：生成的代码清晰可读，便于二次开发
- **移动优先**：针对手机端优化的控件和布局
- **平台集成**：深度集成天枢IoT平台功能

二、功能架构

2.1 整体架构图

```
┌──────────────────────────────────────────────────────────────────┐
│                      天枢IoT Studio (开发工具)                      │
├──────────────────────────────────────────────────────────────────┤
│  项目管理 | 画布编辑 | 代码预览 | 代码导出 | 平台配置同步          |
├──────────────────────────────────────────────────────────────────┤
│                          代码生成引擎                              │
└──────────────────────────────────────────────────────────────────┘
                                ↓ 生成
┌──────────────────────────────────────────────────────────────────┐
│                      生成的Flutter App (移动优先)                  │
├──────────────────────────────────────────────────────────────────┤
│  设备监控页 | 设备控制页 | AI Agent页 | 设置页面                  |
├──────────────────────────────────────────────────────────────────┤
│  本地设置:                                                        │
│  - AI参数配置 (API Key/Base URL/模型/温度等)                      |
│  - 服务器连接配置                                                 |
│  - 界面主题配置                                                   |
├──────────────────────────────────────────────────────────────────┤
│  可选功能:                                                        │
```

```
│   - 用户登录（子账户系统，需平台支持）                                    │
│   - 从平台拉取AI配置                                                      │
├──────────────────────────────────────────────────────────────────────────┤
│                          天枢IoT平台 API                                  │
└──────────────────────────────────────────────────────────────────────────┘
```

三、核心功能模块

3.1 代码生成引擎

生成代码特点

- **可读性优先**：清晰的代码结构和命名
- **注释完整**：关键逻辑有中文注释
- **模块化**：按功能拆分文件，便于维护
- **最佳实践**：遵循Flutter官方推荐模式

生成的项目结构

```
generated_app/
├── lib/
│   ├── main.dart              # 入口文件
│   ├── app.dart               # App配置
│   ├── pages/                 # 页面
│   │   ├── home_page.dart
│   │   ├── device_page.dart
│   │   ├── ai_chat_page.dart
│   │   └── settings_page.dart
│   ├── widgets/               # 自定义控件
│   ├── services/              # API服务
│   │   ├── api_service.dart
│   │   └── ai_service.dart
│   ├── models/                # 数据模型
│   └── config/                # 配置
│       └── app_config.dart
├── pubspec.yaml
└── README.md                  # 使用说明
```

3.2 画布编辑器（开发工具侧）

核心特性

- **移动端预览**：模拟手机屏幕比例
- **拖拽放置**：从控件库拖拽到画布
- **实时预览**：所见即所得
- **代码同步**：编辑时实时生成代码

界面布局

```
┌──────────────────────────────────────────────────────────────────────┐
│ 工具栏: [保存] [预览] [导出代码] [项目设置]                          │
├──────────┬──────────────────────────────┬──────────────────────────┤
│ 控件库   │        手机模拟器预览        │      属性/代码面板       │
│          │     ┌──────────────┐         │                          │
│ [显示]   │     │  生成的App   │         │  [属性] [代码]           │
│ [输入]   │     │   预览效果   │         │                          │
│ [控制]   │     │              │         │  控件属性配置            │
│ [图表]   │     └──────────────┘         │  或实时代码预览          │
│ [布局]   │                              │                          │
└──────────┴──────────────────────────────┴──────────────────────────┘
```

3.3 控件系统（移动端优化）

3.3.1 数据显示控件

控件名称	功能描述	可配置项
数值显示	显示单个数值（温度、湿度等）	单位、精度、颜色阈值
文本显示	显示文本状态	字体、颜色、对齐
状态指示灯	LED样式状态显示	颜色映射、闪烁效果
进度条	百分比进度显示	方向、颜色、范围
仪表盘	圆形仪表显示	刻度、范围、警戒区
图片显示	显示设备图片/图标	缩放、圆角
时间戳	显示最后更新时间	格式、相对时间

3.3.2 数据输入控件

控件名称	功能描述	可配置项
数值输入	输入数值参数	范围、步进、单位
文本输入	输入文本参数	长度限制、正则验证
滑块	滑动选择数值	范围、步进、刻度
下拉选择	枚举值选择	选项列表
颜色选择	RGB/HSV颜色选择	预设颜色
日期时间	日期时间选择	格式、范围

3.3.3 控制控件

控件名称	功能描述	可配置项
开关按钮	布尔值切换	标签、颜色
按钮	触发命令执行	文字、图标、确认框
按钮组	多个互斥按钮	按钮列表
摇杆	方向控制（机器人等）	灵敏度、回弹
调色板	LED灯带颜色控制	预设、渐变

3.3.4 图表控件

控件名称	功能描述	可配置项
折线图	时序数据趋势	时间范围、多线、颜色
柱状图	数据对比	分组、堆叠
饼图	占比分布	标签、颜色
散点图	数据分布	点大小、颜色映射
热力图	二维数据分布	颜色梯度
实时曲线	滚动实时数据	缓冲区大小、刷新率

3.3.5 布局控件

控件名称	功能描述	可配置项
容器	分组控件	边框、背景、标题
分割线	视觉分隔	方向、样式
标签页	内嵌标签切换	标签列表
折叠面板	可展开/收起区域	默认状态

3.4 AI Agent 模块

3.4.1 功能概述

- 独立的AI对话标签页
- 支持多种AI接入方式
- 设备控制和数据查询能力
- 对话历史管理

3.4.2 AI接入方式

模式	描述	优点	缺点
平台在线	直接使用天枢IoT平台配置的AI	无需配置、统一管理	依赖平台
平台代理	通过平台API网关代理AI请求	统一计费、安全	有延迟
本地直连	用户自行配置API Key本机调用	低延迟、灵活	需自行管理

3.4.3 AI工具能力（Function Calling）

基于天枢IoT平台已有的Agent工具系统：

工具	功能	参数
device_control	控制设备	device_id, command, params
device_query	查询设备状态	device_id
data_query	查询历史数据	device_id, limit
config_get	获取配置	device_id
config_set	设置配置	device_id, key, value

3.4.4 AI界面设计

```
┌──────────────────────────────────────────────────────────────┐
│  AI Agent: 智能助手                                [设置] [新建] │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 🤖 你好！我是天枢IoT智能助手，可以帮你：              │    │
│  │     - 查询设备状态和数据                              │    │
│  │     - 控制设备开关和参数                              │    │
│  │     - 分析历史数据趋势                                │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 👤 帮我查看客厅温度传感器的当前数据                   │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 🤖 正在查询设备数据...                                │    │
│  │     📊 客厅温度传感器 (在线)                          │    │
│  │     温度: 24.5°C | 湿度: 65%                          │    │
│  │     最后更新: 2分钟前                                 │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│  [ 🎤 ] 输入消息...                                [发送]     │
└──────────────────────────────────────────────────────────────┘
```

3.5 设置页面

3.5.1 连接设置

配置项	描述	默认值
服务器地址	天枢IoT平台API地址	http://localhost:5000
用户凭证	用户名/密码或Token	-
自动重连	断线自动重连	开启
数据刷新间隔	实时数据轮询间隔	5秒
超时时间	API请求超时	30秒

3.5.2 AI设置

配置项	描述
AI模式	平台在线/平台代理/本地直连
API提供商	OpenAI/Claude/通义千问/自定义
API Key	本地直连时的密钥
Base URL	自定义API端点
模型选择	可用模型列表
温度参数	0-2，控制随机性
最大Token	回复长度限制

3.5.3 界面设置

配置项	描述
主题	亮色/暗色/跟随系统
语言	中文/英文
网格大小	画布网格像素
吸附开关	控件对齐吸附
自动保存	定时保存项目

四、增强功能

4.1 子账户系统（需修改天枢IoT平台）

功能说明

主账户可创建子账户，子账户使用生成的App访问受限资源。

平台端新增API

- `POST /auth/sub-accounts` - 创建子账户
- `GET /auth/sub-accounts` - 子账户列表
- `PUT /auth/sub-accounts/<id>` - 更新子账户
- `DELETE /auth/sub-accounts/<id>` - 删除子账户

子账户权限配置

权限项	描述
可访问设备	指定可查看/控制的设备列表
可用Agent	指定可使用的AI Agent
数据权限	只读/读写
有效期	账户过期时间

4.2 生成App的设置页面（可配置）

设置页面配置项（开发工具中配置）

用户在Studio中可选择生成的App包含哪些设置项：

设置模块	可选	描述
服务器连接	是	服务器地址、超时等
用户登录	是	启用子账户登录功能
AI本地配置	是	API Key、模型、参数等
从平台拉取AI配置	是	使用平台配置的AI
主题设置	是	亮色/暗色/跟随系统
语言设置	是	多语言支持
数据刷新	是	刷新间隔配置
关于页面	是	App版本信息

AI本地配置详情

生成的App中，用户可在设置页面配置：

配置项	描述
API提供商	OpenAI/Claude/通义千问/DeepSeek/自定义
API Key	用户自己的密钥
Base URL	自定义API端点（兼容OpenAI格式）
模型名称	如gpt-4、claude-3等
温度	0-2，控制随机性
最大Token	回复长度限制
系统提示词	可选自定义

4.3 设备配对功能（可选，需修改平台）

使用场景

场景	是否需要配对
DIY自用	否，直接绑定设备ID
产品级应用	是，用户扫码/输入配对码绑定设备

配对流程

1. 设备生成配对码（6位数字，有效期5分钟）
2. 用户在App输入配对码或扫描二维码
3. 平台验证并绑定设备到用户账户

平台端新增API

- `POST /devices/pairing-code` - 设备请求生成配对码
- `POST /devices/pair` - 用户提交配对码绑定设备
- `DELETE /devices/<id>/binding` - 解绑设备

4.4 设备列表功能（可选）

使用场景

场景	是否需要设备列表
单设备控制	否，App直接控制指定设备
多设备用户	是，用户可切换/管理多个设备

功能详情

- 显示用户已绑定的所有设备
- 设备在线/离线状态显示
- 切换当前控制的设备
- 设备重命名、解绑操作

4.5 项目管理

- **项目创建**：新建空白项目或从模板创建
- **项目导入/导出**：JSON格式项目文件
- **云端同步**：与天枢IoT平台同步项目
- **版本历史**：本地版本快照

4.6 模板系统

- **设备模板同步**：从平台拉取设备模板
- **自动生成控件**：根据模板数据点自动生成对应控件
- **命令面板**：根据模板命令定义生成控制按钮

4.7 数据绑定系统

- **可视化绑定**：点选设备和字段完成绑定
- **表达式支持**：简单计算和格式化
- **条件样式**：根据数据值改变控件样式

4.8 告警系统

- **阈值告警**：数据超限提醒
- **离线告警**：设备离线通知
- **告警历史**：告警记录查看

4.9 快捷操作

- **快捷键**：常用操作键盘快捷键
- **右键菜单**：上下文操作菜单
- **命令面板**：Ctrl+P快速命令

4.10 离线模式

- **本地缓存**：设备列表和配置缓存
- **离线编辑**：无网络时编辑项目
- **同步队列**：恢复连接后同步操作

五、技术架构

5.1 技术栈

层级	技术选型
UI框架	Flutter 3.x
状态管理	Riverpod / Bloc
路由	go_router
网络请求	dio
本地存储	hive / shared_preferences
图表库	fl_chart / syncfusion_flutter_charts
拖拽库	flutter_draggable_gridview

5.2 项目结构

```
tianshu_studio/
├── lib/
|   ├── main.dart
|   ├── app.dart
|   ├── core/                 # 核心功能
|   |   ├── config/          # 配置
|   |   ├── constants/       # 常量
|   |   ├── theme/           # 主题
|   |   └── utils/           # 工具类
|   ├── data/                 # 数据层
|   |   ├── models/          # 数据模型
|   |   ├── repositories/    # 数据仓库
|   |   └── services/        # API服务
|   ├── features/             # 功能模块
|   |   ├── canvas/          # 画布编辑器
|   |   ├── widgets/         # 控件库
|   |   ├── ai_agent/        # AI Agent
|   |   ├── settings/        # 设置
|   |   └── project/         # 项目管理
|   └── shared/               # 共享组件
|       ├── widgets/         # 通用控件
|       └── providers/       # 状态提供者
├── assets/                   # 资源文件
├── test/                     # 测试
└── pubspec.yaml
```

5.3 核心数据模型

```
// 项目模型
class Project {
  String id;
  String name;
  List<CanvasPage> pages;
  ProjectSettings settings;
}

// 画布页面
class CanvasPage {
  String id;
  String name;
  List<WidgetInstance> widgets;
}

// 控件实例
class WidgetInstance {
  String id;
  String type;
  Offset position;
  Size size;
  Map<String, dynamic> properties;
  DataBinding? dataBinding;
}

// 数据绑定
class DataBinding {
  String deviceId;
  String fieldName;
  String? expression;
}
```

六、开发阶段

阶段一：基础框架

- 项目初始化和架构搭建
- 多标签页系统实现
- 基础UI框架和主题
- 天枢IoT API对接

阶段二：画布编辑器

- 拖拽系统实现
- 基础控件开发
- 属性面板

- 撤销/重做功能

阶段三：控件系统

- 数据显示控件
- 数据输入控件
- 控制控件
- 图表控件

阶段四：数据绑定

- 设备数据获取
- 实时数据更新
- 数据绑定配置UI

阶段五：AI Agent

- AI对话界面
- 多模式AI接入
- 工具调用集成

阶段六：完善优化

- 设置页面
- 项目导入导出
- 离线模式
- 性能优化

七、API对接清单

7.1 认证相关

- `POST /auth/login` - 登录
- `GET /auth/me` - 获取用户信息

7.2 设备相关

- `GET /devices` - 设备列表
- `GET /devices/<id>` - 设备详情
- `GET /devices/<id>/config` - 获取配置
- `PUT /devices/<id>/config` - 更新配置

7.3 数据相关

- `GET /data/<device_id>` - 历史数据

7.4 模板相关

- `GET /templates` - 模板列表
- `GET /templates/<id>` - 模板详情

7.5 AI Agent相关

- `GET /agents` - Agent列表
- `GET /agents/<id>` - Agent详情
- `POST /agents/<id>/chat` - 对话
- `GET /gateway/configs` - API配置列表

八、总结

天枢IoT Studio 将提供：

1. **代码生成式开发** - 生成可读性高的Flutter源码，支持二次开发
2. **移动端优先** - 针对iOS/Android优化，兼容桌面端
3. **可配置设置页面** - 生成的App设置项可自定义
4. **AI本地配置** - 用户可在App中配置自己的AI API
5. **子账户系统**（可选）- 主账户创建子账户，权限可控
6. **设备配对**（可选）- 产品级应用支持扫码配对
7. **设备列表**（可选）- 多设备用户管理和切换

文档版本: v1.0
创建日期: 2026-01-26